

Axona Topic IDs

How topics are addressed in @axona/protocol v3.2.0 — what a topic ID is, the two kinds (open and owned), how `write` defaults, and how you **generate a topic ID to share** so others can subscribe (and, for open topics, publish).

Companion documents:

- [Quick Start](#) — 5-minute working roundtrip.
- [API Reference](#) — every public symbol.
- [Programmer Guide](#) — mental model + worked example.

The Quick Start / API Reference / Programmer Guide are still on the pre-v3 (v2.40.0) pub/sub surface (string topics + `publishId`). This note reflects the **current** model; where they disagree, this note wins.

1. A topic ID is a hash of a descriptor

A topic is not a bare string. It's a small **descriptor**, and the topic ID is a hash of that descriptor with a one-byte region prefix:

```
topicId = regionByte || SHA-256(canonical({ owner, name, write })) // 33 bytes = 66 hex chars
```

Field	Meaning
<code>region</code>	a real geographic cell; becomes the leading byte of the ID
<code>owner</code>	an Author ID (a publish key), or absent
<code>name</code>	the human label — "lobby", "profile"
<code>write</code>	"open" or "owner" — defaults by whether owner is set (see §2)

Because the ID is a pure function of those fields, **anyone who knows the fields computes the identical ID** — no registry, no coordination.

The `region` field: name or code

`region` accepts a region **name**, **hex string**, **decimal string**, or **number** — all normalize to the same region byte:

```
{ region: 'useast', name: 'lobby' } // name
{ region: '0x89', name: 'lobby' } // hex string --\ same topic ID
{ region: 137, name: 'lobby' } // number --/ (leading byte 0x89)
```

'useast' = '0x89' = 137. Omit `region` and it defaults to the publisher's own node region (top byte of its node ID); name it explicitly to share a topic across regions. Unknown regions throw `RangeError`.

2. `write` defaults by whether there's an owner

Descriptor	Resolved <code>write</code>	Meaning
{ <code>region</code> , <code>name</code> } (no owner)	open	open lobby — anyone publishes
{ <code>region</code> , <code>owner</code> , <code>name</code> } (no <code>write</code>)	owner	owned — only the owner publishes

Descriptor	Resolved write	Meaning
{ region, owner, name, write:'owner' }	owner	same id as the row above
{ region, owner, name, write:'open' }	open	owner-namespaced, anyone publishes (inbox/wall)
{ region, name, write:'owner' } (no owner)	open	no owner -> write ignored

Two rules to remember:

- **No owner -> the topic is open; write is ignored.** You can't have an owner-only topic with no owner.
- **An owner -> write defaults to 'owner'** (owner-only — the safe default). Pass `write:'open'` explicitly only when you want an owner-namespaced topic anyone may post to (an inbox). So `{owner, name}` and `{owner, name, write:'owner'}` are the same topic, and forgetting `write` can never silently make an owned feed world-writable.

3. Open topics — { region, name }

Anyone computes the ID, anyone publishes (each message self-signed by the author you choose, or anonymous), anyone subscribes.

```
import { createAuthorIdentity } from '@axona/protocol';
const me = await createAuthorIdentity();

await peer.pub({ region: 'useast', name: 'lobby' }, 'hello', { signWith: me });
await peer.sub({ region: 'useast', name: 'lobby' }, (env) => {
  console.log(env.signerPubkey, env.message);
}, { since: 'all' });
```

4. Owned topics — { region, owner, name } (write defaults to owner)

Fold your **Author ID** in as owner. Only messages signed by the owner key are accepted — the storing node recomputes the ID from the signed descriptor and rejects anything where `signer !== owner` (WRITE_POLICY_VIOLATION). A profile or feed can't be spoofed even by talking directly to a storage node.

```
const me = await createAuthorIdentity({ persistAs: 'me' });
const feed = { region: 'useast', owner: me.authorId, name: 'profile' }; // write defaults to 'owner'
await peer.pub(feed, { status: 'online' }, { signWith: me }); // only `me` may write
```

Reading stays open — but the ID is *deterministic, not secret*. A stranger can't compute it without the owner's Author ID, so the owner shares the ID (or the descriptor) with readers.

5. Generating a topic ID to share

`deriveTopicId(descriptor)` returns the 66-hex topic ID — the same function the peer uses internally. It works for every topic kind. This is what you hand to someone so they can subscribe.

```
import { deriveTopicId } from '@axona/protocol';

// open lobby
const lobbyId = await deriveTopicId({ region: 'useast', name: 'lobby' });
```

```
// my owned feed (write defaults to 'owner')
const feedId = await deriveTopicId({ region: 'useast', owner: me.authorId, name: 'profile' });

// → e.g. "89a1b2c3..." (66 hex chars; leading "89" is the region byte)
// share lobbyId / feedId out-of-band (URL, QR, message)
```

The **topic ID** is a **read / subscribe handle**. Anyone you give it to can subscribe or pull with the ID alone — no descriptor needed:

```
await peer.sub(lobbyId, onMsg, { since: 'all' }); // sub by ID
await peer.pull/latestMsgId, { topic: feedId }); // pull by ID
await peer.metrics(feedId); // metrics by ID
```

sub, pull, and metrics each accept **either** a descriptor object **or** a 66-hex topic ID.

Publishing needs the descriptor, not the ID

pub (and owner ops kill / unpub) require the **descriptor**, not a bare ID — passing an ID throws. This is by design, not a limitation:

- The storing node must recompute the ID from {region, owner, name, write} to enforce the write policy (signer === owner). A topic ID is a hash; it can't reveal its owner, so the ID **alone cannot prove** you're authorized to write. (If it could, anyone who learned an owned feed's ID could post to it.)
- For an **open** topic the publish descriptor is just {region, name} — tiny and human-meaningful, so “share the topic so someone can post” means sharing those two fields. The owner of an owned topic already holds the full descriptor.

In short: **share the ID for reading; share the descriptor for writing.**

6. Multiple publishers on one owned topic — *proposed, not yet implemented*

Authorizing additional publishers on an owned topic is **not built**: write:'owner' means exactly the one owner key. The intended design (roadmap) is an **owner-signed writer roster**: the topic still commits to one owner in its ID; the owner publishes a signed { topicId, writers:[...], seq } record; storing nodes accept a publish if the signer is the owner **or** is in the latest roster. Adding/removing a writer is a new roster; the topic ID never changes.

7. Quick reference

	Open topic	Owned topic
Descriptor	{ region, name }	{ region, owner, name } (write -> owner)
Who may publish	anyone (self-signed / anonymous)	only the owner key (node-enforced)
Who may subscribe	anyone	anyone given the ID / descriptor
region forms	name · '0x89' · 137, or omit → node region	same
Generate ID to share	deriveTopicId({region, name})	deriveTopicId({region, owner, name})
Read by ID	sub/pull/metrics accept the 66-hex ID	same

	Open topic	Owned topic
Publish by ID	no — needs {region, name} descriptor	no — needs the descriptor (write policy)
Multiple writers	n/a	proposed (owner-signed roster) — not yet built

Two factories, one signer rule: `createNodeIdentity({lat,lng})` is the connection identity; `createAuthorIdentity({persistAs?})` is a location-free publish identity. Every signed publish names its signer via `signWith` (an author, or the `ANONYMOUS` sentinel) — no default signer.